

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/390341702>

Gas Detector With MQ-2 , LED, LCD and Buzzer

Article · March 2025

CITATIONS

0

2 authors:



Fatemeh Nabidoust

Pishgaman IEA Co, LTD

21 PUBLICATIONS 0 CITATIONS

SEE PROFILE



M. Nabidoust

15 PUBLICATIONS 0 CITATIONS

SEE PROFILE

Gas Detector With MQ-2 , LED, LCD and Buzzer

in Arduino

Fatemeh Nabidoust - Electrical and Electronic and Computer Science Department

Mohammad Nabidoust - Electrical and Electronic and Computer Science Department

ABSTRACT

The evolution of gas detection has changed considerably over the years. New, innovative ideas from canaries to portable monitoring equipment provides workers with continuous precise gas monitoring. Gas detection equipment can be broken down into the monitoring of gas using sensors and gas path technology, the user interface that informs people or equipment of any necessary action, and the supporting power management system that keeps it all charged up and working.

1.INTRODUCTION - What is GAS Detector

A gas detector is a device that detects the presence of gases in a volume of space, often as part of a safety system. A gas detector can sound an alarm to operators in the area where the leak is occurring, giving them the opportunity to leave.

This type of device is important because there are many gases that can be harmful to organic life, such as humans or animals.

Gas detectors can be used to detect combustible, flammable and toxic gases, and oxygen depletion. This type of device is used widely in industry and can be found in locations, such as on oil rigs, to monitor manufacturing processes and emerging technologies such as photovoltaic. They may be used in firefighting. Gas leak detection is the process of identifying potentially hazardous gas leaks by sensors. Additionally a visual identification can be done using a thermal camera These sensors usually employ an audible alarm to alert people when a dangerous gas has been detected. Exposure to toxic gases can also occur in operations such as painting, fumigation, fuel filling, construction, excavation of contaminated soils, land-fill operations, entering confined spaces, etc. Common sensors include combustible gas sensors, photoionization detectors, infrared point sensors, ultrasonic sensors, electrochemical gas sensors, and metal-oxide-semiconductor (MOS) sensors.

More recently, infrared imaging sensors have come into use. All of these sensors are used for a wide range of applications and can be found in industrial plants, refineries, pharmaceutical manufacturing, fumigation facilities, paper pulp mills, aircraft and shipbuilding facilities, hazmat operations, waste-water treatment facilities, vehicles, indoor air quality testing and homes.

1.2.Types

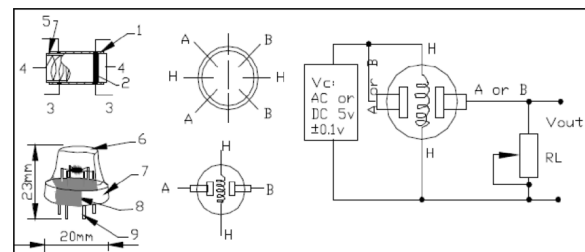
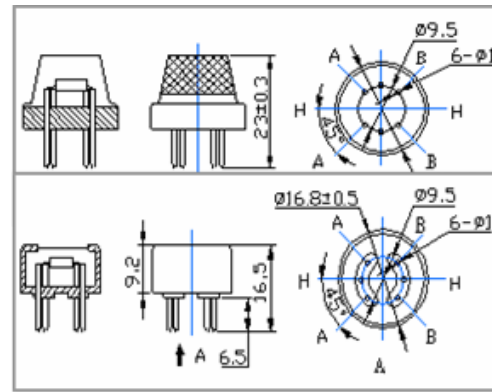
Gas detectors can be classified according to the operation mechanism (semiconductor, oxidation, catalytic, photoionization, infrared, etc.). Gas detectors come packaged into two main form factors: portable devices and fixed gas detectors. Portable detectors are used to monitor the atmosphere around personnel and are either hand-held or worn on clothing or on a belt/harness. These gas detectors are usually battery operated. They transmit warnings via audible and visible signals, such as alarms and flashing lights, when dangerous levels of gas vapors are detected. Fixed type gas detectors may be used for detection of one or more gas types. Fixed type detectors are generally mounted near the process area of a plant or control room, or an area to be protected, such as a residential bedroom. Generally, industrial sensors are installed on fixed type mild steel structures and a cable connects the detectors to a supervisory control and data acquisition (SCADA) system for continuous monitoring. A tripping interlock can be activated for an emergency situation.

2.MQ-2 Gas Sensor

In this Article, I will be explaining you about the MQ2 sensor and how to interface it with Arduino development board.

This Gas sensor is suitable for sensing LPG, Smoke, Alcohol, Propane, Hydrogen, Methane and Carbon Monoxide concentrations in the air. MQ2 is one of the commonly used gas sensors in MQ sensor series. It is a Metal Oxide Semiconductor (MOS) type Gas Sensor also known as Chemiresistors as the detection is based upon change of resistance of the sensing material when the Gas comes in contact with the material. Using a simple voltage divider network, concentrations of gas can be detected.

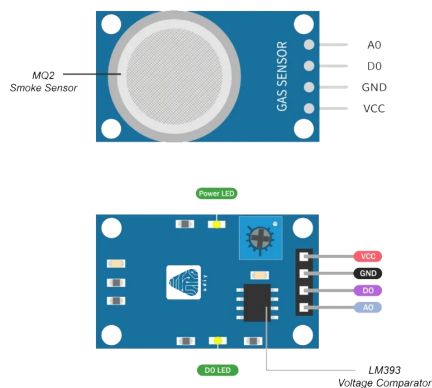
Sensitive material of MQ-2 gas sensor is SnO₂, which with lower conductivity in clean air. When the target combustible gas exist, The sensor's conductivity is more higher along with the gas concentration rising. Please use simple electro circuit, Convert change of conductivity to correspond output signal of gas concentration. MQ-2 gas sensor has high sensitivity to LPG, Propane and Hydrogen, also could be used to Methane and other combustible steam, it is with low cost and suitable for different application.



2.2.Technical Data

Model No.		MQ-2	
Sensor Type		Semiconductor	
Standard Encapsulation		Bakelite (Black Bakelite)	
Detection Gas		Combustible gas and smoke	
Concentration		300-10000ppm (Combustible gas)	
Circuit	Loop Voltage	V_c	$\leq 24V$ DC
	Heater Voltage	V_H	$5.0V \pm 0.2V$ AC or DC
	Load Resistance	R_L	Adjustable
Character	Heater Resistance	R_H	$31\Omega \pm 3\Omega$ (Room Tem.)
	Heater consumption	P_H	$\leq 900mW$
	Sensing Resistance	R_s	$2K\Omega - 20K\Omega$ (in 2000ppm C ₂ H ₆)
	Sensitivity	S	$R_s(\text{in air})/R_s(1000ppm \text{ isobutane}) \geq 5$
	Slope	α	$\leq 0.6 (R_{5000ppm}/R_{3000ppm} CH_4)$
Condition	Tem. Humidity	$20^\circ C \pm 2^\circ C$; $65\% \pm 5\% RH$	
	Standard test circuit	$V_c: 5.0V \pm 0.1V$; $V_H: 5.0V \pm 0.1V$	
	Preheat time	Over 48 hours	

ADIY MQ2 Smoke Sensor Module



2.1.Character

- * Good sensitivity to Combustible gas in wide range
- * High sensitivity to LPG, Propane and Hydrogen
- * Long life and low cost
- * Simple drive circuit

2.3.Application

- * Domestic gas leakage detector
- * Industrial Combustible gas detector
- * Portable gas detector

The above is basic test circuit of the sensor. The sensor need to be put 2 voltage, heater voltage (VH) and test voltage (VC) . VH used to supply certified working temperature to the sensor, while VC used to detect voltage (VRL) on load resistance (RL) whom is in series with sensor. The sensor has light polarity, Vc need DC power. VC and VH could use same power circuit with precondition to assure performance of sensor. In order to make the sensor with better performance, suitable RL value is needed:

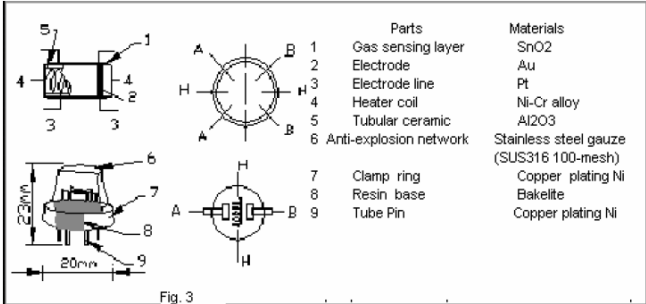
Power of Sensitivity body (Ps):

$$Ps = Vc^2 \times Rs / (Rs + RL)^2$$

Resistance of Sensor (Rs):

$$Rs = (\frac{Vc}{VRL} - 1) \times RL$$

2.4.Structure and Configuration



Structure and configuration of MQ-2 gas sensor is shown as Fig. 3, sensor composed by micro AL2O3 ceramic tube, Tin Dioxide (SnO2) sensitive layer, measuring electrode and heater are fixed into a crust made by plastic and stainless steel net. The heater provides necessary work conditions for work of sensitive components. The enveloped MQ 2 have 6 pin, 4 of them are used to fetch signals, and other 2 are used for providing heating current.

3.How a design a gas detector using the MQ-2 sensor with an Arduino

3.1.Hardware Requirements

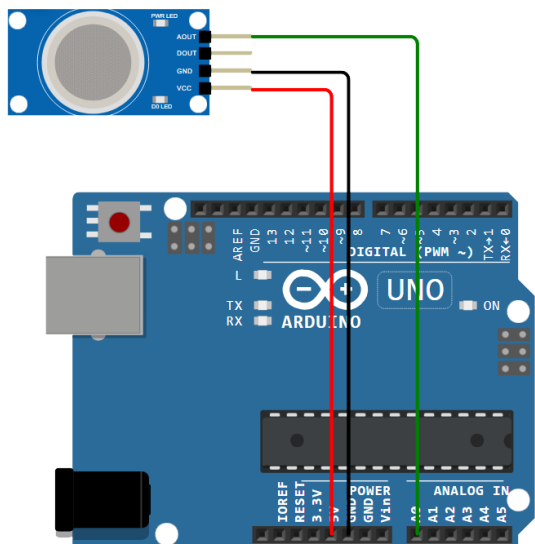
- Arduino Board (e.g., Arduino Uno)
- MQ-2 Gas Sensor
- Jumper Wires
- Breadboard (optional, for easier connections)
- 10kΩ Resistor (optional, for a pull-down resistor if needed)
- Power Supply (Arduino can power the MQ-2 via 5V)
- Concentration Scope: 200 – 10000ppm
- Preheat Time: Over 24 hours

3.2.Pin Connections

The MQ-2 sensor typically has four pins: VCC, GND, DO (Digital Output), and AO (Analog Output). For this basic design, we'll use the analog output to measure gas concentration.

MQ-2 Pin		Arduino Pin	
VCC	5V		
GND	GND		
AO	A0		
DO	(Not used in this example, but can connect to a digital pin if needed)		

- 1.Connect VCC of the MQ-2 to the 5V pin on the Arduino.
- 2.Connect GND of the MQ-2 to the GND pin on the Arduino.
- 3.Connect AO (Analog Output) of the MQ-2 to the A0 pin on the Arduino.



3.3.How It Works

- The MQ-2 sensor outputs an analog voltage proportional to the gas concentration it detects.
- The Arduino reads this voltage via its analog pin (A0) and converts it to a value between 0 and 1023.
- You can set a threshold to trigger an alert (e.g., print a message) when gas levels exceed a certain value.

3.4.Arduino Code

Here's a simple sketch to read the MQ-2 sensor and display the values on the Serial Monitor:

```
// Define the analog pin connected to MQ-2
const int sensorPin = A0;
// Threshold value for gas detection (adjust based on
testing)
const int threshold = 400;
void setup() {
  // Initialize serial communication at 9600 baud rate
  Serial.begin(9600);
  Serial.println("Gas Detector Starting...");

  // Allow sensor to warm up (MQ-2 needs ~20-30 seconds)
  delay(20000); // 20 seconds warm-up time
  Serial.println("Sensor Ready!");
}

void loop() {
  // Read the analog value from the sensor
  int sensorValue = analogRead(sensorPin);

  // Print the sensor value to Serial Monitor
  Serial.print("Gas Sensor Value: ");
  Serial.println(sensorValue);

  // Check if gas concentration exceeds threshold
  if (sensorValue > threshold) {
    Serial.println("WARNING: High Gas Concentration De-
tected!");
  } else {
    Serial.println("Gas Levels Normal");
  }

  // Delay for readability
  delay(1000); // Wait 1 second before next reading
}
```

3.5.Steps to Implement

- 1.Connect the Hardware: Wire the MQ-2 to the Arduino as described above.
- 2.Upload the Code: Open the Arduino IDE, paste the code, and upload it to your Arduino board.
- 3.Test the Setup: Open the Serial Monitor (Ctrl+Shift+M in Arduino IDE) at 9600 baud to see the sensor readings.
- 4.Calibrate: The MQ-2 sensor's sensitivity varies with temperature, humidity, and gas type. You may need to adjust the **threshold** value by exposing the sensor to a known gas source (e.g., a lighter, safely) and noting the readings.

Notes

- **Warm-Up Time:** The MQ-2 requires a warm-up period (20-30 seconds or more) to stabilize its readings.
- **Ventilation:** Test in a well-ventilated area to avoid false positives.
- **Enhancements:** You can add an LED or buzzer to pin D2 (for example) to create a visual/audible alarm (Article Part 2)

4.Adding LED in Project

I assume you want the LED to turn on when the sensor value crosses the threshold and off when the gas is at normal.

4.1.Hardware Connections

- LED Anode (longer leg): Connect to Arduino pin D2.
- LED Cathode (shorter leg): Connect through a 220Ω or 330Ω resistor to Arduino GND.

LED	Arduino Pin
Anode	D2
Cathode	GND(with resistor)

```
// Define the analog pin connected to MQ-2 @elec_astro
const int sensorPin = A0;
// Define the LED pin
const int ledPin = 2; // Pin D2 for the LED
// Threshold value for gas detection (adjust based on testing)
const int threshold = 400;
void setup() {
  // Initialize serial communication at 9600 baud rate
  Serial.begin(9600);
  Serial.println("Gas Detector Starting...");

  // Set LED pin as output
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW); // LED initially off

  // Allow sensor to warm up (MQ-2 needs ~20-30 seconds)
  delay(20000); // 20 seconds warm-up time
  Serial.println("Sensor Ready!");
}
void loop() {
  // Read the analog value from the sensor
  int sensorValue = analogRead(sensorPin);

  // Print the sensor value to Serial Monitor
  Serial.print("Gas Sensor Value: ");
  Serial.println(sensorValue);

  // Check if gas concentration exceeds threshold
  if (sensorValue > threshold) {
    Serial.println("WARNING: High Gas Concentration Detected!");
    digitalWrite(ledPin, HIGH); // Turn on the LED
  } else {
    Serial.println("Gas Levels Normal");
    digitalWrite(ledPin, LOW); // Turn off the LED
  }

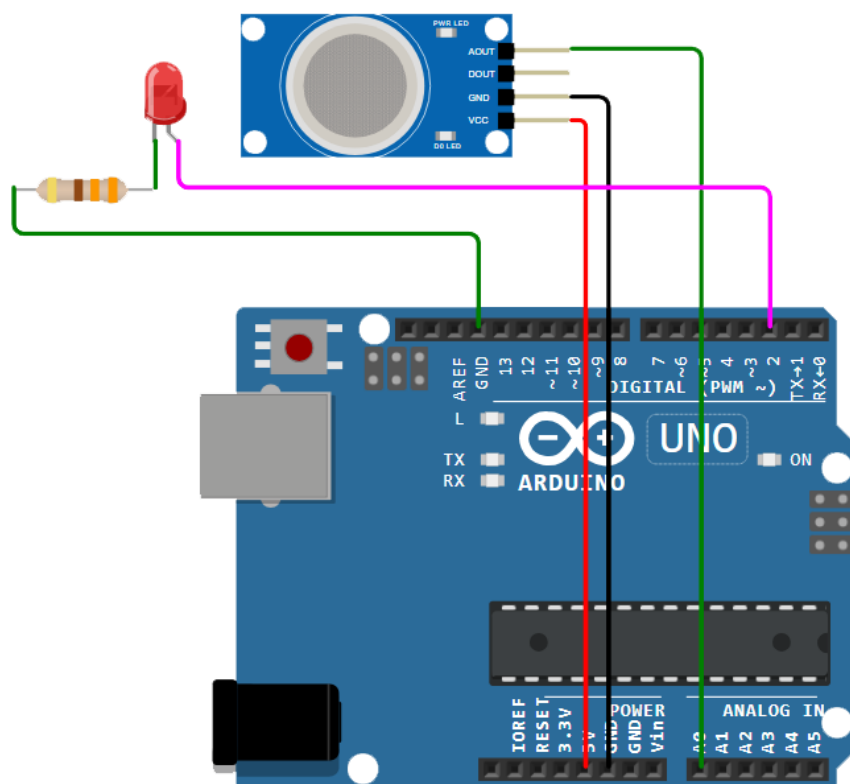
  // Delay for readability
  delay(1000); // Wait 1 second before next reading
}
```

4.2.Explanation

- LED Pin Definition: The **ledPin** is set to D2 (you can change it to any other digital pin if needed).
- Initial Setup: In **setup()**, the LED pin is configured as an output and turned off initially.
- LED Control:
 - If **sensorValue** exceeds **threshold**, the LED turns on (**HIGH**).
 - If the value is below the threshold, the LED turns off (**LOW**).
- Behavior: The LED updates its state every time the sensor is read (every 1 second), turning on or off based on the gas level.

4.3.Testing

- Connect the LED with a resistor to the Arduino as shown in the table above.
- Upload the code to your Arduino.
- Open the Serial Monitor (9600 baud).
- Expose the sensor to gas (e.g., carefully use a lighter). When the value exceeds 400, the LED will turn on, and it will turn off when the the gas level returns to normal.



5.Adding Buzzer in Project

Gas sensors are vital components in ensuring the safety of indoor environments by detecting hazardous gases such as carbon monoxide or smoke. When paired with an Arduino board and a buzzer, these sensors can serve as early warning systems, alerting users to the presence of harmful gases. When we apply bias to the sensor, it takes some time for the sensor to warm, after that the electrochemical sensor detects specific Gas and varies the current flow through the sensor. Hence we get analog output ranges depends on Gas concentration.

5.1.Components Needed

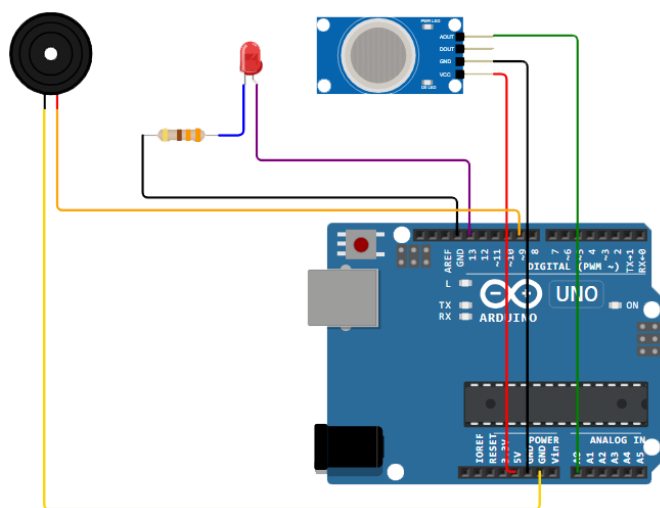
- Arduino Board (e.g., Arduino Uno)
- MQ-2 Gas Sensor
- Breadboard (optional, for easy connections)
- Jumper Wires
- 330Ω Resistor (optional, for a voltage divider if needed)
- LED (optional, for visual indication)
- Buzzer (optional, for sound alerts)
- Power Supply (USB or external 5V for Arduino)

5.2.Wiring the MQ-2 to Arduino

The MQ-2 sensor typically has 4 pins: VCC, GND, DO (Digital Output), and AO (Analog Output). For this project, we'll use the analog output to measure gas concentration.

MQ-2 Pin	Arduino Pin	Description
VCC	5V	Power supply (5V)
GND	GND	Ground connection
AO	A0	Analog output to Arduino
DO	optional	Digital output (not used here)

- ⇒ Connect VCC of the MQ-2 to the 5V pin on the Arduino.
- ⇒ Connect GND of the MQ-2 to the GND pin on the Arduino.
- ⇒ Connect AO (Analog Output) of the MQ-2 to the A0 pin on the Arduino.
- ⇒ (Optional) Add an LED to pin 13 and GND (with a 220Ω resistor in series) for visual alerts.
- ⇒ (Optional) Add a buzzer to pin 9 and GND for sound alerts.



```
// Define pins @elec_astro
const int sensorPin = A0; // MQ-2 analog output pin
const int ledPin = 13;    // LED pin (optional)
const int buzzerPin = 9;  // Buzzer pin (optional)
const int threshold = 300; // Adjust this threshold based on your calibration
void setup() {
  // Initialize serial communication
  Serial.begin(9600);

  // Set pin modes
  pinMode(sensorPin, INPUT);
  pinMode(ledPin, OUTPUT);
  pinMode(buzzerPin, OUTPUT);

  Serial.println("Gas Detector Starting...");
  delay(20000); // Allow sensor to warm up for 20 seconds
}
void loop() {
  // Read the analog sensor value
  int sensorValue = analogRead(sensorPin);

  // Print the value to Serial Monitor
  Serial.print("Sensor Value: ");
  Serial.println(sensorValue);

  // Check if gas level exceeds threshold
  if (sensorValue > threshold) {
    digitalWrite(ledPin, HIGH); // Turn on LED
    tone(buzzerPin, 1000);      // Activate buzzer at 1000 Hz
    Serial.println("Gas Detected!");
  } else {
    digitalWrite(ledPin, LOW); // Turn off LED
    noTone(buzzerPin);         // Turn off buzzer
    Serial.println("No Gas Detected");
  }

  delay(1000); // Wait 1 second before next reading
}
```

5.3.How It Works

1.Warm-Up Time: The MQ-2 sensor needs about 20-30 seconds to stabilize after powering on, so we include a delay in **setup()**.

2.Sensor Reading: The **analogRead()** function reads the voltage on pin A0, which corresponds to gas concentration (0-1023 range).

3.Threshold: You'll need to adjust the **threshold** value based on your environment and sensor calibration. Test it in clean air first to determine a baseline (usually 100-200), then set the threshold higher (e.g., 300).

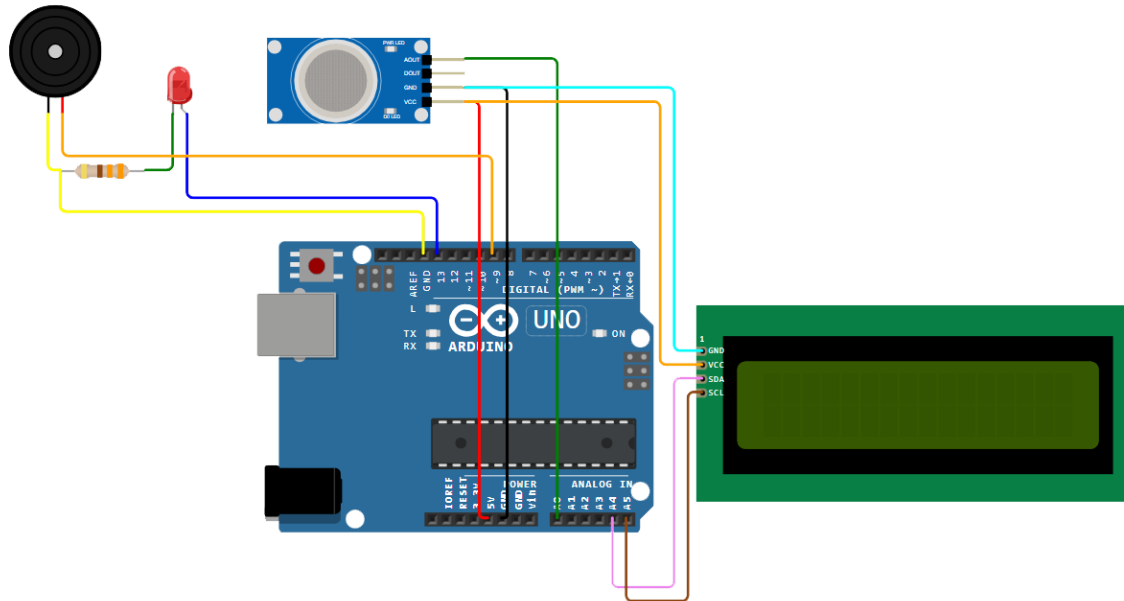
4.Alerts: If the sensor value exceeds the threshold, the LED lights up and the buzzer sounds.

5.4.Calibration Tips

- * Run the sensor in clean air and note the typical analog value (e.g., 100-200).
- * Expose it to a small amount of gas (e.g., lighter gas, safely) to see how the value changes.
- * Adjust the **threshold** in the code accordingly.

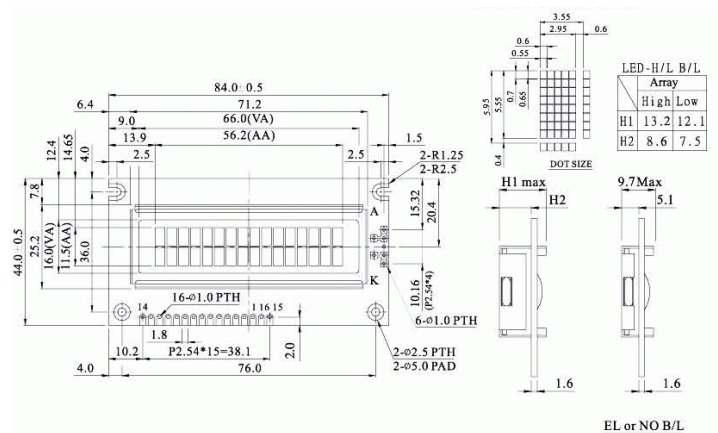
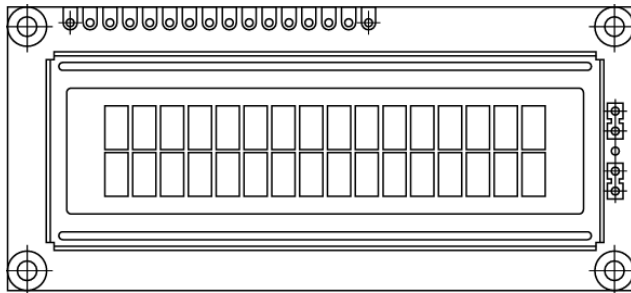
6.Adding LCD 16 x 2 with the I2C extender

I'll assume you're using a common 16x2 LCD with an I2C interface (like the HD44780 with an I2C backpack) since it simplifies wiring.



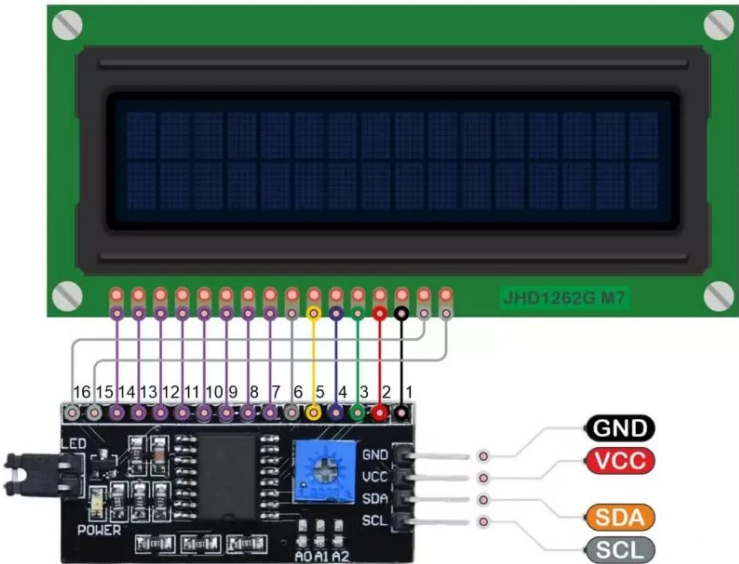
6.1.Updated Components

- 16x2 LCD with I2C Interface



6.2.Wiring the LCD with I2C

The I2C interface reduces the number of pins needed. It typically has 4 pins: VCC, GND, SDA, and SCL.



- GND** is a ground pin and should be connected to the ground of Arduino.
- VCC** supplies power to the module and the LCD. Connect it to the 5V output of the Arduino or a separate power supply.
- SDA** is a Serial Data pin. This line is used for both transmit and receive. Connect to the SDA pin on the Arduino.
- SCL** is a Serial Clock pin. This is a timing signal supplied by the Bus Master device. Connect to the SCL pin on the Arduino.

6.3.In Project:

LCD Pin	Arduino Pin	Description
VCC	5V	Power supply (5V)
GND	GND	Ground connection
SDA	A4	Data line (on Uno)
SCL	A5	Clock line (on Uno)

Note: If you’re using a different Arduino (e.g., Mega), SDA and SCL pins might differ (e.g., 20 and 21 on Mega). Check your board’s pinout.

Updated Wiring

- Keep the MQ-2, LED, and buzzer wired as before.
- Add the LCD connections as table above.

6.4.Required Library

You'll need the **LiquidCrystal_I2C** library to control the LCD. Install it in the Arduino IDE:

- 1.Go to Sketch > Include Library > Manage Libraries.
- 2.Search for "LiquidCrystal_I2C" by Frank de Brabander.
- 3.Install the library.

```
#include <Wire.h>           // Library for I2C communication @elec_astro
#include <LiquidCrystal_I2C.h> // Library for I2C LCD
// Define pins
const int sensorPin = A0; // MQ-2 analog output pin
const int ledPin = 13;    // LED pin
const int buzzerPin = 9;  // Buzzer pin
const int threshold = 300; // Adjust this threshold based on calibration
// Initialize LCD (address 0x27 is common, but check yours with an I2C scanner if needed)
LiquidCrystal_I2C lcd(0x27, 16, 2);
void setup() {
  // Initialize serial communication (optional, for debugging)
  Serial.begin(9600);

  // Set pin modes
  pinMode(sensorPin, INPUT);
  pinMode(ledPin, OUTPUT);
  pinMode(buzzerPin, OUTPUT);

  // Initialize LCD
  lcd.init();
  lcd.backlight(); // Turn on backlight
  lcd.setCursor(0, 0);
  lcd.print("Gas Detector");
  lcd.setCursor(0, 1);
  lcd.print("Warming Up...");

  Serial.println("Gas Detector Starting...");
  delay(20000); // Allow sensor to warm up for 20 seconds

  lcd.clear(); // Clear the "Warming Up" message
}
void loop() {
  // Read the analog sensor value
  int sensorValue = analogRead(sensorPin);

  // Display on LCD
  lcd.setCursor(0, 0); // First line
  lcd.print("Value: ");
  lcd.print(sensorValue);
  lcd.print(" "); // Clear extra characters

  lcd.setCursor(0, 1); // Second line
  if (sensorValue > threshold) {
    lcd.print("Gas Detected! ");
    digitalWrite(ledPin, HIGH); // Turn on LED
    tone(buzzerPin, 1000);      // Activate buzzer
  } else {
    lcd.print("No Gas ");
    digitalWrite(ledPin, LOW);  // Turn off LED
    noTone(buzzerPin);          // Turn off buzzer
  }

  // Optional: Print to Serial Monitor for debugging
  Serial.print("Sensor Value: ");
  Serial.println(sensorValue);

  delay(1000); // Update every 1 second
}
```

6.5.How It Works

1.LCD Setup: The **LiquidCrystal_I2C** library initializes the LCD with its I2C address (commonly 0x27 or 0x3F—use an I2C scanner sketch if it doesn't work).

2.Display:

- First line shows the sensor value (e.g., "Value: 150").
- Second line shows "Gas Detected!" or "No Gas" based on the threshold.

3.Clearing Extra Characters: The `print(" ")` ensures old text is overwritten, keeping the display clean.

6.6.I2C Address Troubleshooting

If the LCD doesn't display anything:

Run this I2C Scanner to find your LCD's address.

Replace **0x27** in **LiquidCrystal_I2C lcd(0x27, 16, 2)** with the correct address.

